MAJOR PROJECT ASSIGNMENT

Database Management System

Using Python & SQLite

DIFFICULTY	LANGUAGE	DATABASE
★★★★☆ Intermediate	**Python 3.x**	**SQLite3**

INTERFACE

Command Line (CLI)

Objective

Design and develop a database-driven application using Python and SQLite. The project should demonstrate your understanding of database design, SQL queries, CRUD operations, user authentication, role-based authorization, relationships between tables, and exception handling.

The application must be fully functional through the command line. Students may use Procedural Programming, Object-Oriented Programming (OOP), or a combination of both.

Suggested Project Topics

Choose one of the following, or propose your own topic with instructor approval.

1. Student Management System	**2.** Library Management System	**3.** Hospital Management System
4. Inventory Management System	**5.** Hotel Booking System	**6.** Employee Management System
7. Payroll Management System	**8.** Vehicle Rental System	**9.** Restaurant Management System
10. Online Food Ordering System	**11.** Pharmacy Management System	**12.** Banking Management System
13. Gym Membership Management System	**14.** Event Management System	**15.** Cinema Ticket Booking System
16. Online Shopping Management System	**17.** Warehouse Management System	**18.** Courier Management System
19. School Management System	**20.** Clinic Appointment Management System	**21.** Examination Management System
22. Hostel Management System	**23.** Bus Reservation System	**24.** Tourism Management System
25. Complaint Management System	**26.** Task Management System	**27.** Customer Relationship Management (CRM)
28. Pet Care Management System	**29.** Property Rental Management System	**30.** Course Enrollment Management System

Project Requirements

1 SQLite Database

- Use SQLite as the database.
- The database should be created automatically if it does not already exist.

2 Database Design

Your project must contain a **minimum of 2 related tables** (3 or more recommended). The database must demonstrate:

- Primary Key
- Foreign Key
- Relationships between tables

3 User Authentication

Implement a login system. The system must support:

- User Registration
- User Login
- Logout

4 Role-Based Authorization

Implement at least two user roles — e.g. Administrator, Staff, Manager, Customer, Employee. Each role must have different permissions. For example:

- Admin can perform all CRUD operations.
- Staff/User should have limited access based on the project requirements.

5 CRUD Operations

Implement complete CRUD functionality. Your application must allow users to:

- Create records
- View records
- Update records
- Delete records

CRUD operations should be available only to users with appropriate permissions.

6 Relationships Between Tables

The project must demonstrate relationships between tables using Foreign Keys. The application should correctly maintain data consistency between related tables.

7 Menu-Driven Interface

Create a clear command-line menu system. The application should include:

- Main Menu
- Login Menu
- Role-specific Menus
- Exit Option

Navigation should be simple and user-friendly.

8 Input Validation

Validate all user input. The program should prevent:

- Empty inputs
- Invalid data types
- Duplicate entries
- Invalid IDs
- Invalid menu selections

9 Exception Handling

Use proper exception handling throughout the application. Handle errors such as:

- Invalid user input
- SQLite database errors
- Duplicate records
- Missing records
- Unexpected runtime errors

The program should continue running without crashing due to invalid input or database errors.

10 SQL Operations

Your project must make use of SQLite queries including:

- CREATE TABLE
- INSERT
- SELECT
- UPDATE
- DELETE

Technical Requirements

Your project should demonstrate the use of:

Variables & Data Types	Functions	Conditional Statements
Loops	Lists, Tuples, Dicts & Sets	SQLite3 Module
SQL Queries	CRUD Operations	Primary & Foreign Keys
User Authentication	Role-Based Authorization	Exception Handling
File Handling (if required)	OOP (Optional)	

Submission Requirements

Students must submit:

- Complete Python source code
- SQLite database (.db) file
- README file with setup instructions
- Sample database records (if applicable)
- Well-commented and properly formatted code

Mandatory Checklist

✓ SQLite Database	✓ Minimum 2 related tables
✓ Primary Key & Foreign Key relationship	✓ User Registration
✓ User Login	✓ Role-Based Access Control
✓ Complete CRUD Operations	✓ Input Validation
✓ Exception Handling	

Evaluation Rubric (100 Marks)

Criteria	Marks
Database Design & Table Relationships	**20**
CRUD Operations	**25**
User Authentication & Role-Based Authorization	**20**
SQLite Implementation	**15**
Exception Handling & Input Validation	**10**
Code Quality, User Interface & Documentation	**10**
Total	**100**